

Part 1: Linear Regression & Statistical Modelling

Objective:

Analyse relationships between variables and build predictive models using Scikit-learn and Statsmodels.

Tasks:

1. Load and prepare your dataset (handle missing values and data types).

```
df = pd.read_csv("kpoprates_stats_sheet.csv")

#print(df.dtypes) # Check the column headers and data types

df.drop(["Artist", "Year", "Unnamed: 4", "R#", "Rate", "/", "Unnamed: 10", "11s", "≥10", "≥9", "<5", "<3", "0s", "P#", "Unnamed: 18", "11%", "≥10%", "<3%", "0%", "Unnamed: 25", "Bonus", "Date", "ID", "CU", "ArtistList", "Win"], axis = 1, inplace = True) # Drop all columns I have no interest in
df["≥9%"] = df["≥9%"].str.replace("%", "").astype(float)
df["<5%"] = df["<5%"].str.replace("%", "").astype(float)
print(df.dtypes)
```

2. Perform descriptive statistics: mean, variance, standard deviation.

3. Conduct correlation analysis and interpret relationships.

```
scoreColumn = df["Score"]
controversyColumn = df["Controv."]
highScoresCol = df["≥9%"]
lowScoresCol = df["<5%"]
#print(scoreColumn.type())

print(scoreColumn.count(), highScoresCol.count()) # Checking the dimensions of the data array
averageScore = float("{:.3f}".format((scoreColumn.mean())))
averageControversy = float("{:.2f}".format((controversyColumn.mean())))
scoreVariance = float("{:.3f}".format((scoreColumn.var())))
controversyVariance = float("{:.2f}".format((controversyColumn.var())))
scoreStd = float("{:.3f}".format((scoreColumn.std())))
controversyStd = float("{:.2f}".format((controversyColumn.std())))
```

```

print(f"Average score: {averageScore}\nScore variance:
{scoreVariance}\nScore standard deviation: {scoreStd}\nAverage
controversy: {averageControversy}\nControversy variance:
{controversyVariance}\nControversy standard deviation:
{controversyStd}")

correlationHighScores =
float("{:.2f}".format((scoreColumn.corr(highScoresCol))))
correlationLowScores =
float("{:.2f}".format((scoreColumn.corr(lowScoresCol))))

print(f"Correlation between average score and high scores:
{correlationHighScores}\nCorrelation between average score and low
scores: {correlationLowScores}")

```

The results:

Average score: 7.032
 Score variance: 0.677
 Score standard deviation: 0.823

Average controversy: 2.13
 Controversy variance: 0.13
 Controversy standard deviation: 0.36

Correlation between average score and high scores: 0.83
 Correlation between average score and low scores: -0.89

What do the results signify?

As we can see in the code above, I checked the correlation between the average score of each song and the number of 'high scores' (higher than or equal to 9) they received, as well as the number of 'low scores' they received (lower than 5). There is a very strong positive correlation with the high scores of 0.83, and inversely a very strong negative correlation with the number of low scores of -0.89. This is the expected result, as large numbers of very high/low scores in either direction would play a significant role in the final average of a song's score.

The standard deviation of 0.823 compared to the average score of 7.032 is more than 10%, indicating that whilst there is a reasonable spread of averages, the results themselves are largely consistent, with most songs not deviating from between a score of 6 and 8. The low variance of 0.677 further indicates this.

4. Build a Linear Regression model using:

- Scikit-learn
- Statsmodels (include regression summary)

```
model2 = sm.OLS(lowScoresCol, scoreColumn)
res = model2.fit()
print(res.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  <5%   R-squared (uncentered):              0.581
Model:                          OLS   Adj. R-squared (uncentered):          0.581
Method:                        Least Squares   F-statistic:                        3076.
Date:                Thu, 30 Apr 2026   Prob (F-statistic):                  0.00
Time:                      10:00:49   Log-Likelihood:                     -8833.8
No. Observations:                2219   AIC:                               1.767e+04
Df Residuals:                    2218   BIC:                               1.768e+04
Df Model:                          1
Covariance Type:                nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
Score          2.1560        0.039     55.463      0.000         2.080         2.232
=====
Omnibus:                 542.900   Durbin-Watson:                   0.446
Prob(Omnibus):            0.000   Jarque-Bera (JB):                 1282.894
Skew:                     1.345   Prob(JB):                         2.65e-279
Kurtosis:                 5.577   Cond. No.                         1.00
=====

```

5. Make predictions using your model.

Evaluation:

- R^2 Score
- Mean Squared Error
- Interpretation of coefficients and p-values (Statsmodels)

```
mae = mean_absolute_error(y_test, results["Predicted"])
mse = mean_squared_error(y_test, results["Predicted"])
r2 = r2_score(y_test, results["Predicted"])
```

```

MAE: 3.272526300593346
MSE: 17.505924747221545
R2: 0.8617130504472454

```

Validation:

- Cross-validation (K-Fold)

```
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
```

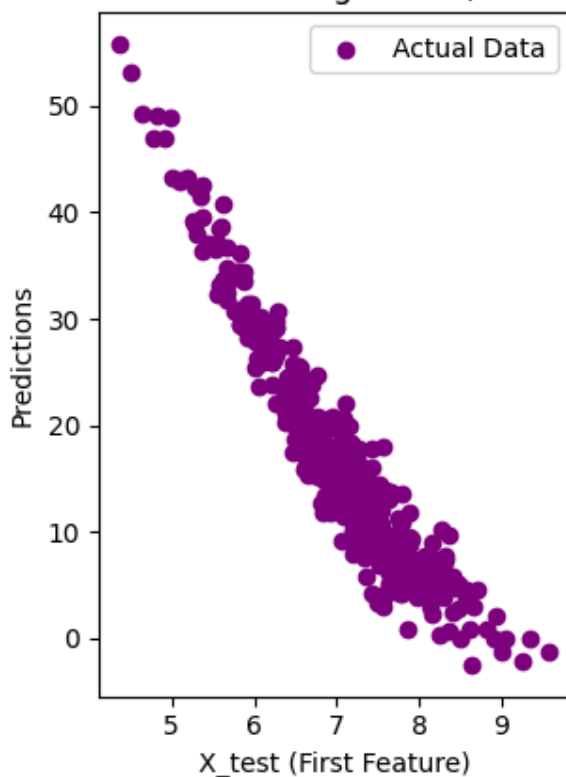
```
kScores = cross_val_score(model, X_test, y_test, cv=kf)
print(f"The scores for each K fold: {kScores}")
print(f"The average K Fold score: {np.mean(kScores)}")
```

```
The scores for each K fold: [0.79921549 0.85834638 0.89031869 0.8367349 0.89881751]
The average K Fold score: 0.8566865949755584
```

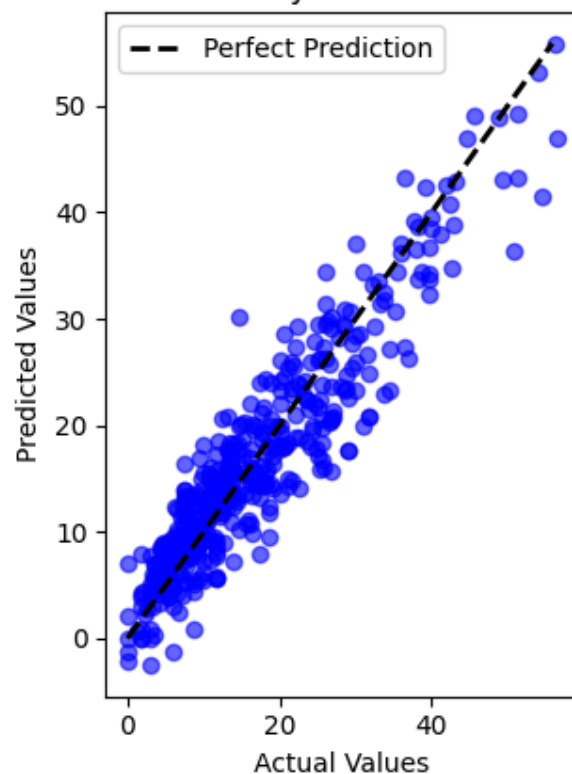
Output:

- Scatter plot with regression line
- Interpretation of model performance and significance

The model relationship (Scores below 5 vs average score)



Accuracy of the model



Part 2: Clustering Analysis

Objective:

Group similar data points using clustering techniques.

Tasks:

1. Select 2–4 numerical features from your dataset.
2. Perform descriptive statistics (mean, standard deviation).

```
# By checking for all rows that contain NaN values, we can determine
what is the necessary course of action to take for each one.
# We can see that the bottom row is used to contain league averages.
For Rk, I will change the value to the median. For the string values
(Team, Pos), I will assign them the name League Average.
# For all the other numeric values in that row i.e. the rest of the
values, I will calculate the mean value in each column and assign it as
league average
# We can see that every other row with a NaN value is in the 3P%
column, where the 3PA per game is 0; based on this info, and what we
can infer from both the positions of all the players being Centre and
what we know about those players and their playstyles is that these
players haven't attempted any 3-pointers this season. Using this
knowledge, I will change those NaN values to 0, as that is the most
suitable adaptation for our purposes.
```

```
# =====
# Calculating and inserting median rank for Rk column
# Replacing string values with "League Average"
# =====

df["Rk"][242] = float("{:.1f}".format((df["Rk"].median())))
df["Team"][242] = "League Average"
df["Pos"][242] = "League Average"
```

```
# =====
# Calculating means for each column
# =====

ptsMean = float("{:.1f}".format((pts.mean())))
minutesPlayedMean = float("{:.1f}".format((minutesPlayed.mean())))
threesMadeMean = float("{:.1f}".format((threesMade.mean())))
threesAttemptedMean = float("{:.1f}".format((threesAttempted.mean())))
threePercentageMean = float("{:.1f}".format((threePercentage.mean())))
totalReboundsMean = float("{:.1f}".format((totalRebounds.mean())))
assistsMean = float("{:.1f}".format((assists.mean())))
stealsMean = float("{:.1f}".format((steals.mean())))
blocksMean = float("{:.1f}".format((blocks.mean())))
turnoversMean = float("{:.1f}".format((turnovers.mean())))
```

```
# =====
# Inserting means into the league average row
# =====
```

```

leagueAvgRow = ["PTS", "MP", "3P", "3PA", "TRB", "AST", "STL", "BLK",
"TOV"]
leagueAvgMeans = [ptsMean, minutesPlayedMean, threesMadeMean,
threesAttemptedMean, totalReboundsMean, assistsMean, stealsMean,
blocksMean, turnoversMean]

i = 0

for x in leagueAvgRow :

    df[x][242] = leagueAvgMeans[i]
    i += 1

i = 0

print(df.loc[[242]])

# =====
# Replacing the NaN values in the 3P% column with 0
# Checking if there are any null values remaining
# =====

df = df.fillna(float("{:.1f}".format(0)))
df1 = df[df.isna().any(axis = 1)]
print(df1)

```

```

print(f"Average points per game: {ptsMean}\nPoints per game variance:
{ptsVar}\nPPG standard deviation: {ptsStd}\nAverage 3 point percentage:
{threePercentageMean}\n3P% variance: {threePercentageVar}\n3P% standard
deviation: {threePercentageStd}")

```

```

Average points per game: 11.9
Points per game variance: 39.32448151549162
PPG standard deviation: 6.270923497818453
Average 3 point percentage: 0.3
3P% variance: 0.009210765214903811
3P% standard deviation: 0.0959727316215591
Correlation between points scored and threes attempted: 0.6221792281066445
Correlation between points and total rebounds: 0.46239113441456237
Correlation between threes attempted and three point percentage: 0.2647420487977607
Correlation between threes made and three point percentage: 0.31928312340988096
Correlation between threes made and threes attempted: 0.9868016007956941

```

```

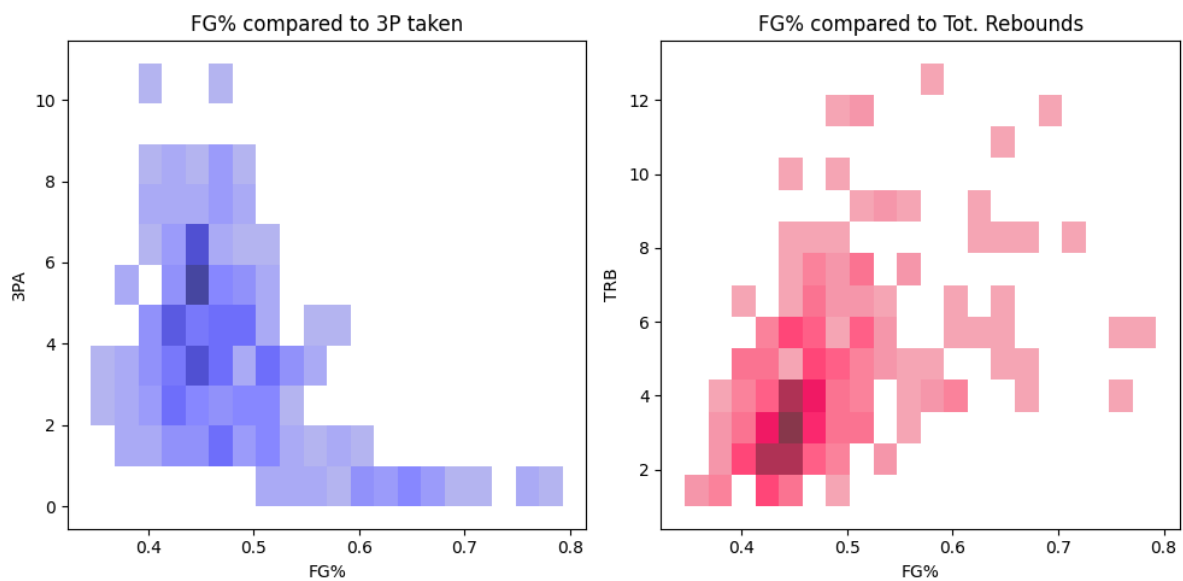
plt.figure(figsize=(10,5))

```

```
plt.subplot(1, 2, 1)
sns.histplot(data = df, x="FG%", y="3PA", kde=True, color="blue")
plt.title("FG% compared to 3P taken")

plt.subplot(1, 2, 2)
sns.histplot(data = df, x="FG%", y="TRB", kde=True, color="pink")
plt.title("FG% compared to Tot. Rebounds")

plt.tight_layout()
plt.show()
```



3. Apply K-Means clustering using Scikit-learn.

```
X = df[["PTS", "FG%"]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(X)
print(X_scaled)

kmeans = KMeans(n_clusters=5, init = "k-means++", random_state=42)

clusters = kmeans.fit_predict(X_scaled)
df["Cluster"] = clusters

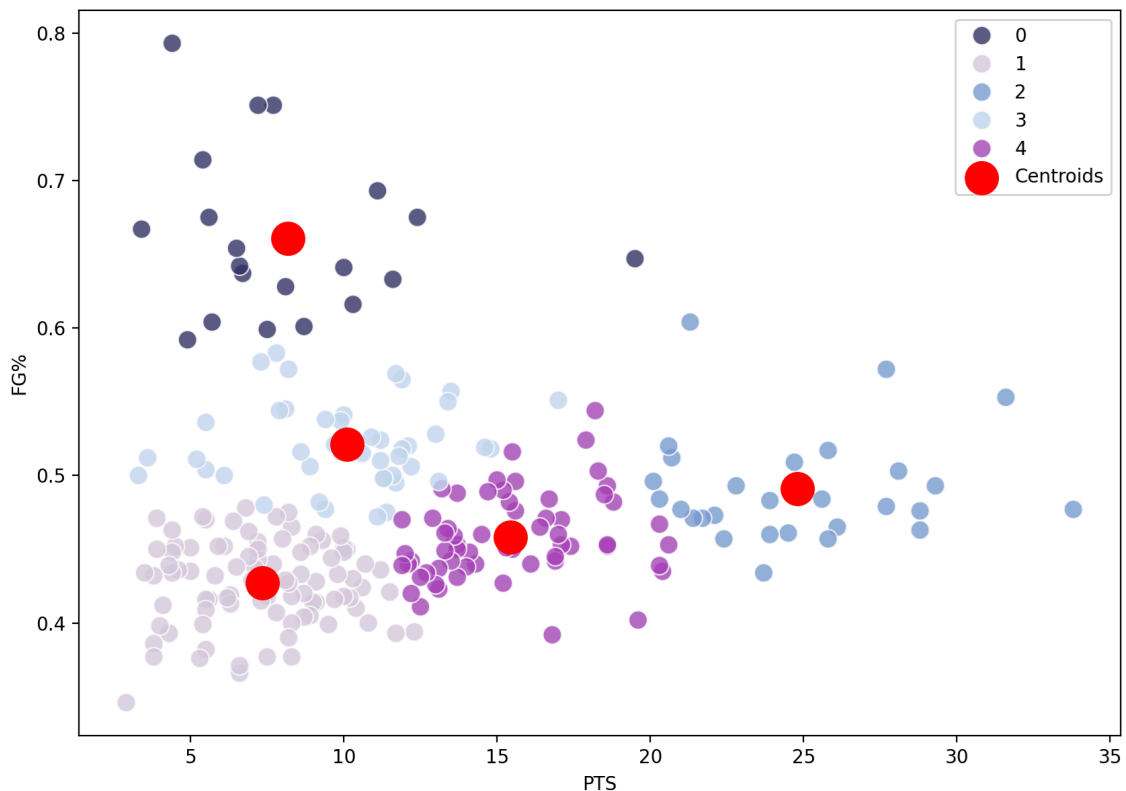
plt.figure(figsize=(10, 7))
sns.scatterplot(data = df, x="PTS", y="FG%", hue="Cluster", s=100,
alpha=0.8, palette=sns.color_palette(colours, len(colours)))

centres = scaler.inverse_transform(kmeans.cluster_centers_)
```

```
plt.scatter(centres[:, 0], centres[:, 1], s=300, c="red", marker="o",
label = "Centroids")
plt.legend()

plt.show()

target_group = df[df["Cluster"] == 1]
print(f"Number of target group: {len(target_group)}")
```



4. Determine optimal number of clusters using the Elbow Method.

```
wcss = []

for i in range(1, 11) :

    kmeans = KMeans(n_clusters=i, init="k-means++", random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker = 'x', linestyle = "--")
plt.show()
```

5. Visualise clusters using scatter plots.

```
colours = ["#333366", "#d6cadd", "#789bd1", "#c1d7ee", "#a743ba"]

plt.subplot(2, 2, 1)

plt.title("Points scored against shooting percentage")
sns.scatterplot(data = df, x = "PTS", y="FG%", hue = "Cluster",
palette=sns.color_palette(colours, len(colours)))

plt.subplot(2, 2, 2)

plt.title("Guard labels")
for idx, row in dfG5.iterrows(): # USE WHEN I WANT TO SHOW PLAYER NAMES
    TO ILLUSTRATE VALIDITY OF DATA

    plt.annotate(row['Player'], (row['PTS'], row['FG%']),
xytext=(15,180), textcoords="offset pixels", arrowprops=dict(width =
0.2, headwidth = 3, headlength = 5, facecolor='black', shrink=0.05),
fontsize = 7)

sns.scatterplot(data = df, x = "PTS", y="FG%", hue = "Cluster",
palette=sns.color_palette(colours, len(colours)))

plt.subplot(2, 2, 3)

plt.title("Forwards labels")
for idx, row in dfF5.iterrows(): # USE WHEN I WANT TO SHOW PLAYER NAMES
    TO ILLUSTRATE VALIDITY OF DATA

    plt.annotate(row['Player'], (row['PTS'], row['FG%']),
xytext=(15,180), textcoords="offset pixels", arrowprops=dict(width =
0.2, headwidth = 3, headlength = 5, facecolor='black', shrink=0.05),
fontsize = 7)

sns.scatterplot(data = df, x = "PTS", y="FG%", hue = "Cluster",
palette=sns.color_palette(colours, len(colours)))

plt.subplot(2, 2, 4)

plt.title("Centers labels")
```

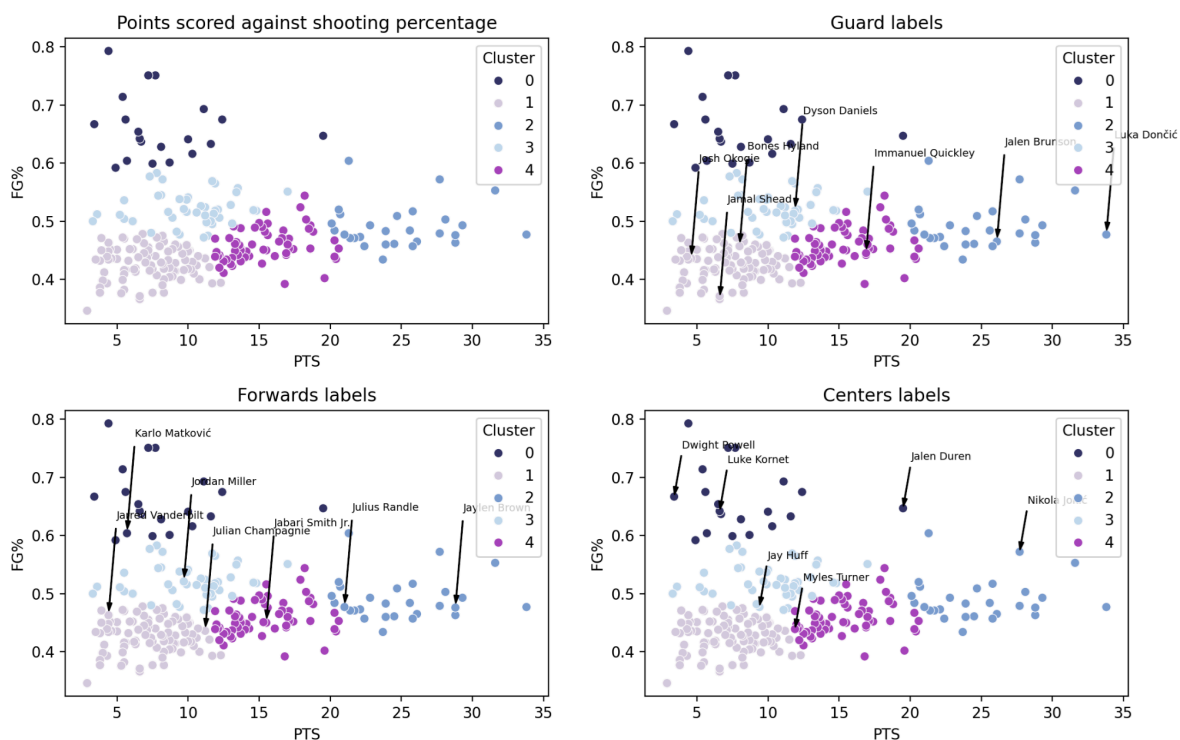
```

for idx, row in dfC5.iterrows(): # USE WHEN I WANT TO SHOW PLAYER NAMES
    TO ILLUSTRATE VALIDITY OF DATA

    plt.annotate(row['Player'], (row['PTS'], row['FG%']),
xytext=(15,90), textcoords="offset pixels", arrowprops=dict(width =
0.2, headwidth = 3, headlength = 5, facecolor='black', shrink=0.05),
fontSize = 7)

sns.scatterplot(data = df, x = "PTS", y="FG%", hue = "Cluster",
palette=sns.color_palette(colours, len(colours)))

```



Interpretation:

- Explain cluster characteristics
- Identify patterns and trends

From this data, we can see 5 clear clusters emerge:

1. Players who score few points, inefficiently
2. Players who score few points, somewhat efficiently
3. Players who score few points, very efficiently
4. Players who score a moderate amount of points, with moderate efficiency
5. Players who score a lot of points, with moderate efficiency

We can see that the majority of players who score a moderate amount of points or more usually regress to a somewhat average level of efficiency, regardless of how many points they score. The most variance between efficiency happens for players with a low amount of

points scored. By using our prior knowledge of basketball, we can likely identify the groups of players that each of these clusters belong to.

I would expect that high efficiency, low point scorers tend to be average players in the Center or Power Forward position. This is because those players are often positioned much closer to the basket than other positions, and so take shots that are easier to score, increasing their shooting percentages. I expect them to be average players, rather than poor players despite their low points scored, because their total volume of shots taken is low, indicating that other players around them take more shots, and so these players are only given the ball in high percentage positions. The type of player structure just described is generally a team that would be considered good, so to take minutes on those teams, you are unlikely to be a poor player. As we can see from the Center labelled graph, These are players such as Dwight Powell and Luke Kornet, who fit that criteria.

I expect the low efficiency, low points scorers to be poor players on below average teams, and are likely to be guards. This is because guards generally take shots further away from the basket with a higher degree of difficulty, and require good spacing provided by the rest of the team in order to make those shots easier. Therefore, being on a poor team would make this more difficult, lowering efficiency. Jamal Shad and Josh Okogie demonstrate this.

The players that score moderate points with moderate efficiency are probably the most diverse group of players. I expect this group to have several archetypes in it, such as 'floor spacing Centers' who are able to shoot further from the basket and have large value in the modern NBA (Myles Turner), guards and forwards who are good players on poor teams (Immanuel Quickley), and also guards and forwards who are average players on good teams (Jabari Smith Jr.).

The last cluster of players who score a high amount of points per game are the players who are often considered team leaders, and are often labelled as 'stars'. We can see some of the best players in the league in this category, such as Nikola Jokic, Jalen Brunson, Jaylen Brown, and Luka Doncic. Of these players, we can see that Nikola Jokic has the highest efficiency; this makes sense based on what we discussed earlier, as he is a Center, and often takes shots from closer to the basket.

By analysing this data and seeing who falls where on the graphs, you can check the value of a player's production against the efficiency with which their player archetype is expected to perform. For example, we can see Jalen Duren in the Centers graph is performing well above expectations, with significantly more points scored than the other high efficiency players. Conversely, we can see that a player like Julius Randle in the forwards section, who generally plays very close to the basket and has been an All Star before, is not scoring as many points as the others in his cluster, and is also less efficient than you would expect from a player so close to the basket. This can inform teams and players during contract negotiations, giving both sides extra leverage as they compare their value vs others of similar roles within the league.